Switzerland's Export Destinations

Annual partner-total discovery, 2023–2025

Table of contents

1 Summary and V2 scope **1**

2 Requirements and configuration **2**
- 2.1 Reproduction options . . . 2
- 2.2 API key . . . 3
- 2.3 V2 partner-total query settings . . . 3
- 2.4 Readable V2 query settings . . . 5

3 ELT process **5**
- 3.1 Extract: request one year of partner totals . . . 5
- 3.2 Extract: collect the partner summaries . . . 7
- 3.3 Inspect and validate the partner discovery records . . . 8
- 3.4 Save the validated V2 partner-summary snapshot . . . 10
- 3.5 Understanding the V2 discovery data . . . 11
- 3.6 Readable V2 partner-summary CSV . . . 14
- 3.7 Database setup: connect to Atlas . . . 15
- 3.8 V2 discovery interpretation and next steps . . . 21
- 3.9 Top 30 partner countries chosen from above . . . 25
- 3.10 Extract HS4 detail for the selected partners . . . 25
- 3.11 Load HS4 records and rank headings in MongoDB . . . 34

4 Sources and method references **42**

5 PDF export and final submission checklist **42**

1 Summary and V2 scope

This V2 notebook is a separate branch of the Swiss Comtrade project. It begins by collecting Switzerland's annual export totals by partner area for 2023–2025. The original `UN_commodity.ipynb` remains the World-by-chapter baseline.

The partner discovery query covers Switzerland (Comtrade reporter code **757**), annual goods exports, H6 classification, and all partner areas. It returns each reported partner area and the World aggregate. The next MongoDB stage will rank partner areas; after the user authorizes that stage, a later request will retrieve HS4 detail for the selected top 30.

Planned analysis after those stages:

1. Which partner areas have the largest total export values across 2023–2025?
2. Which HS4 headings have the largest export values within those top 30 partners?
3. How do partner and product shares change across the three years?
4. How concentrated are exports across partner areas and HS4 headings?

`primaryValue` is in current US dollars. It measures trade value, not physical volume. Partner areas may include territories or special aggregates, not only sovereign countries.

Current V2 workflow: request partner totals → check and save the discovery data.

Next stage: the cells below support HS4 extraction, snapshot saving, raw loading, and top-50 ranking. New API and database operations require explicit opt-in. The nested clean collection and research-question analysis remain future work; unexecuted cells do not establish results.

2 Requirements and configuration

Use a Python kernel with `requests`, `pandas`, `python-dotenv`, `pymongo`, and `dnspython` installed. Install the repository-local dependencies with `python -m pip install -r requirements.txt` from this notebook's folder. Use the same Python environment as the notebook kernel. `python-dotenv` loads local configuration; `pymongo` connects to MongoDB, and `dnspython` supports Atlas SRV address lookup.

The course template uses `requests.get()`, `.json()`, a list of records, and simple loops with a delay. V2 keeps those patterns. Discovery uses `partnerCode=None`, which omits the partner filter, and `cmdCode="TOTAL"`. HS4 extraction later uses individual partner codes and `cmdCode="AG4"`.

```
import os
import time
import json
import math
from datetime import datetime, timezone
from getpass import getpass
from pathlib import Path
from pprint import pprint

import pandas as pd
import pymongo
import requests
from dotenv import load_dotenv

# Keep notebook tables compact enough for reading and later PDF export.
pd.set_option("display.max_rows", 20)
pd.set_option("display.max_colwidth", 60)
```

2.1 Reproduction options

Option A — fresh partner discovery: leave `DATA_SOURCE = "api"`. V2 requests the annual partner totals and saves a dated snapshot after validation.

Option B — saved partner discovery snapshot: set `DATA_SOURCE = "snapshot"` and set `SNAPSHOT_PATH` to the exact V2 partner-summary JSON file. No API key or network request is needed in this mode; the query scope and record structure are checked before using its records.

V2 snapshots use a different query scope from the V1 World-by-chapter snapshots. Do not reuse a V1 snapshot as V2 partner data.

```
DATA_SOURCE = "snapshot" # snapshot or api
SNAPSHOT_PATH = Path("data/v2_partner_totals_20261016T143600945247Z.json")

if DATA_SOURCE not in {"api", "snapshot"}:
    raise ValueError("Choose either api or snapshot as DATA_SOURCE.")
```

2.2 API key

Only needed for fresh extraction; skipped in snapshot mode. Use the registered UN Comtrade key already configured for this project. Keep the key in the local `.env` file or use the hidden prompt; do not copy it into the notebook, outputs, snapshots, or CSV files.

V2 uses the authenticated `/data/v1/get` endpoint. See the official UN Comtrade API documentation for current access and parameter behavior.

```
headers = {}
# Read the secret without writing it into the notebook source.
if DATA_SOURCE == "api":
    load_dotenv(dotenv_path=".env", override=False)
    api_key = os.environ.get("UN_COMTRADE_API_KEY", "").strip()
    if not api_key:
        api_key = getpass("UN Comtrade API key: ").strip()

    if not api_key:
        raise ValueError("An API key is required for this authenticated endpoint.")

    # A header keeps the key out of the query URL.
    headers = {"Ocp-Apim-Subscription-Key": api_key}
    print("API key loaded. Its value is not displayed.")
else:
    print("Snapshot mode: no API key is loaded or requested.")
```

```
Snapshot mode: no API key is loaded or requested.
```

2.3 V2 partner-total query settings

Setting	Meaning
`C / A / HS` in the endpoint	Goods, annual frequency, original HS classification
`reporterCode = 757`	Switzerland; Comtrade code, not ISO numeric code 756
`partnerCode = None`	Omit the partner filter; return reported partner areas plus World
`cmdCode = "TOTAL"`	Total across commodity codes; this discovery stage has no HS4 breakdown

Setting	Meaning
`flowCode = "X"`	Exports
`partner2Code = 0`, `customsCode = "C00"`, `motCode = 0`	Keep these additional dimensions at the project's aggregate selections
`maxRecords = 100000`	Free registered API per-call limit; V2 checks the response count and stops if a query reaches the cap

The partner summary is a discovery dataset, not the final partner-by-product dataset. The `ALL` result includes a World row; keep it for context, but exclude it from partner rankings. Some partner-area codes may represent geographic or special aggregates.

2.3.1 API parameter reference

Parameter	Meaning	V2 value
`typeCode`	Goods (`C`) or services (`S`)	`C` in the URL
`freqCode`	Annual (`A`) or monthly (`M`)	`A` in the URL
`clCode`	Product classification	`HS` in the URL; returned edition must be `H6`
`period`	Calendar year	One year per request, 2023–2025
`reporterCode`	Country/area reporting trade	`757`: Switzerland
`partnerCode`	Partner country/area	`None`: omit this filter and include World
`cmdCode`	Commodity selection	`TOTAL`: aggregate all commodity codes for destination discovery
`flowCode`	Trade direction	`X`: exports
`partner2Code`	Additional aggregate dimension	`0`
`customsCode`, `motCode`	Customs and transport selections	`C00`, `0`

These are discovery totals. HS4 codes are requested only after the top partner areas have been selected in the later MongoDB phase.

```
API_URL = "https://comtradeapi.un.org/data/v1/get/C/A/HS"
YEARS = ["2023", "2024", "2025"]
MAX_RECORDS = 100000
REQUEST_DELAY = 5
EXPECTED_EDITION = "H6"

# These filters describe one annual all-commodity total per partner area.
base_params = {
    "reporterCode": 757,
    "partnerCode": None,
    "partner2Code": 0,
```

```
    "cmdCode": "TOTAL",
    "flowCode": "X",
    "customsCode": "C00",
    "motCode": 0,
    "maxRecords": MAX_RECORDS,
    "includeDesc": "true"
}
```

2.4 Readable V2 query settings

Export one row of query parameters per year for provenance. These are request settings, not trade observations; they contain no API key.

```
Path("data").mkdir(exist_ok=True)
query_csv = Path("data") / f"v2_partner_query_settings_{datetime.now(timezone.utc):%Y%m%dT%H
pd.DataFrame([{**base_params, "period": year} for year in YEARS]).to_csv(query_csv, index=Fa
print(f"V2 query settings saved to {query_csv}")
```

```
V2 query settings saved to data/v2_partner_query_settings_20261017T092929813727Z.csv
```

3 ELT process

ELT means **Extract, Load, Transform**: collect source data, load it in raw form, then transform it in the target database. V2 first extracts partner-total discovery records. After MongoDB connection is authorized, aggregation pipelines will rank partners; a later API request will retrieve HS4 detail for the selected partners.

3.1 Extract: request one year of partner totals

`requests.get()` sends the query. `.raise_for_status()` stops on an HTTP error, and `.json()` parses the response into Python dictionaries and lists. The API response includes a `count`; V2 checks it against the number of records in `data`. The `ALL` partner selection includes the World aggregate.

```
def request_comtrade_records(year, query_params, request_headers):
    params = query_params.copy()
    params["period"] = year
    for attempt in range(3):
        try:
            response = requests.get(
                API_URL, headers=request_headers, params=params, timeout=45
            )
        except requests.exceptions.Timeout:
            if attempt == 2:
                raise
            wait_seconds = 15 * (attempt + 1)
            print(f"Timeout for {year}; retrying in {wait_seconds} seconds.")
```

```
            time.sleep(wait_seconds)
            continue

        if response.status_code == 429 and attempt < 2:
            wait_seconds = 15 * (attempt + 1)
            print(f"Rate limit for {year}; retrying in {wait_seconds} seconds.")
            time.sleep(wait_seconds)
            continue

        if response.status_code == 400:
            try:
                error_payload = response.json()
                if isinstance(error_payload, dict):
                    error_detail = error_payload.get("error") or error_payload.get("message"
                else:
                    error_detail = error_payload
            except ValueError:
                error_detail = response.text[:500]
            raise ValueError(f"Comtrade rejected the {year} query: {error_detail}")

        response.raise_for_status()
        payload = response.json()

        if not isinstance(payload, dict):
            raise ValueError("Expected a JSON object from Comtrade.")
        if payload.get("error"):
            raise ValueError("Comtrade reported an API error; review the query and documenta

        if not isinstance(payload.get("data"), list):
            raise ValueError("Expected a list of records in the API's data field.")

        records = payload["data"]
        count = payload.get("count")
        if isinstance(count, bool) or not isinstance(count, (int, str)):
            raise ValueError("Expected an integer record count from Comtrade.")
        try:
            api_count = int(count)
        except (TypeError, ValueError):
            raise ValueError("Expected an integer record count from Comtrade.") from None
        if api_count != len(records):
            raise ValueError("Comtrade's response count differs from its data list.")
        if api_count >= MAX_RECORDS:
            raise ValueError("The record cap was reached; partition the query before continu

        return records, api_count


def extract_partner_totals(year):
    return request_comtrade_records(year, base_params, headers)
```

3.2 Extract: collect the partner summaries

The loop makes one data request per year. `list_trade` holds the raw partner-area records; `extraction_log` records each year's API count, received count, query, and timestamp. This cell does not write to MongoDB. The later MongoDB stage will rank the partners.

```
list_trade = []
extraction_log = []
query_scope = {
    "endpoint": API_URL,
    "years": YEARS,
    "filters": base_params,
    "expected_edition": EXPECTED_EDITION,
    "partner_selection": "ALL",
    "commodity_selection": "TOTAL"
}

if DATA_SOURCE == "snapshot":
    with SNAPSHOT_PATH.open(encoding="utf-8") as file:
        snapshot = json.load(file)
    if not isinstance(snapshot, dict) or snapshot.get("schema_version") != 2:
        raise ValueError("Unsupported V2 partner-summary snapshot format.")
    if snapshot.get("query_scope") != query_scope:
        raise ValueError("Snapshot query settings differ from this V2 notebook's settings.")
    if not isinstance(snapshot.get("records"), list):
        raise ValueError("Snapshot must contain a list of raw partner records.")
    if not isinstance(snapshot.get("extraction_log"), list):
        raise ValueError("Snapshot must contain an extraction log.")
    list_trade = snapshot["records"]
    extraction_log = snapshot["extraction_log"]
    print(f"Loaded {len(list_trade)} partner-total records from {SNAPSHOT_PATH}.")
else:
    for i, year in enumerate(YEARS):
        print(f"Checking partner totals for {year} ({i + 1}/{len(YEARS)})")
        extracted_at = datetime.now(timezone.utc).isoformat()
        records, api_count = extract_partner_totals(year)

        list_trade.extend(records)
        extraction_log.append({
            "year_requested": year,
            "api_count": api_count,
            "records_received": len(records),
            "extracted_at_utc": extracted_at,
            "endpoint": API_URL,
            "query": {**base_params, "period": year}
        })

        if i < len(YEARS) - 1:
            time.sleep(REQUEST_DELAY)

    print(f"Partner discovery finished: {len(list_trade)} raw records.")
```

```
if not list_trade:
    raise ValueError("No partner-total records are available to inspect.")
```

```
Loaded 673 partner-total records from data/v2_partner_totals_20261016T143600945247Z.json.
```

3.3 Inspect and validate the partner discovery records

Each record is one year's reported Swiss export total for a partner area. `partnerCode = 0` is the World aggregate, not an individual partner; retain it for context but exclude it from partner rankings. Partner reference codes can also represent areas or aggregates. This discovery extract has no HS4 breakdown yet.

```
# Inspect a raw trade document, not the secret-bearing request headers.
pprint(list_trade[0])
```

```
{'aggrLevel': 0,
 'altQty': 0.0,
 'altQtyUnitAbbr': 'N/A',
 'altQtyUnitCode': -1,
 'cifvalue': None,
 'classificationCode': 'H6',
 'classificationSearchCode': 'HS',
 'cmdCode': 'TOTAL',
 'cmdDesc': 'All Commodities',
 'customsCode': 'C00',
 'customsDesc': 'TOTAL CPC',
 'flowCode': 'X',
 'flowDesc': 'Export',
 'fobvalue': 419922543423.509,
 'freqCode': 'A',
 'grossWgt': 0.0,
 'isAggregate': True,
 'isAltQtyEstimated': False,
 'isGrossWgtEstimated': False,
 'isLeaf': False,
 'isNetWgtEstimated': False,
 'isOriginalClassification': True,
 'isQtyEstimated': False,
 'isReported': False,
 'legacyEstimationFlag': 0,
 'mosCode': '0',
 'motCode': 0,
 'motDesc': 'TOTAL MOT',
 'netWgt': None,
 'partner2Code': 0,
 'partner2Desc': 'World',
 'partner2ISO': 'W00',
 'partnerCode': 0,
 'partnerDesc': 'World',
```

```
 'partnerISO': 'W00',
 'period': '2023',
 'primaryValue': 419922543423.509,
 'qty': 0.0,
 'qtyUnitAbbr': 'N/A',
 'qtyUnitCode': -1,
 'refMonth': 52,
 'refPeriodId': 20230101,
 'refYear': 2023,
 'reporterCode': 757,
 'reporterDesc': 'Switzerland',
 'reporterISO': 'CHE',
 'typeCode': 'C'}
```

```python
expected_fields = {
    "reporterCode": base_params["reporterCode"],
    "typeCode": "C",
    "freqCode": "A",
    "cmdCode": base_params["cmdCode"],
    "flowCode": base_params["flowCode"],
    "partner2Code": base_params["partner2Code"],
    "customsCode": base_params["customsCode"],
    "motCode": base_params["motCode"]
}

if sorted(entry.get("year_requested") for entry in extraction_log) != sorted(YEARS):
    raise ValueError("Extraction log does not cover each requested year exactly once.")

if sum(entry.get("records_received", 0) for entry in extraction_log) != len(list_trade):
    raise ValueError("Extraction-log record counts do not match the collected records.")
if any(
    entry.get("api_count") != entry.get("records_received")
    for entry in extraction_log
):
    raise ValueError("An API response count differs from the received record count.")

actual_keys = []
for record in list_trade:
    year = str(record.get("period"))
    if year not in YEARS:
        raise ValueError("Unexpected year in partner summary.")
    for field, expected in expected_fields.items():
        if record.get(field) != expected:
            raise ValueError(f"Unexpected value for {field}; review the response.")
    if not record.get("partnerCode") and record.get("partnerCode") != 0:
        raise ValueError("A record is missing its partner code.")
    if record.get("classificationCode") != EXPECTED_EDITION:
        raise ValueError("Unexpected HS edition; check comparability.")
    value = record.get("primaryValue")
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise ValueError("A record is missing a numeric primary trade value.")
```

```
    if not math.isfinite(value) or value < 0:
        raise ValueError("A record has a non-finite or negative trade value.")
    actual_keys.append((year, str(record["partnerCode"])))

if len(actual_keys) != len(set(actual_keys)):
    raise ValueError("Duplicate year/partner records found.")

for year in YEARS:
    world_rows = [
        record for record in list_trade
        if str(record.get("period")) == year and str(record.get("partnerCode")) == "0"
    ]
    if len(world_rows) != 1:
        raise ValueError(f"Expected one World aggregate record for {year}.")

print(f"Validated {len(list_trade)} partner-total records across {len(YEARS)} years.")
print("The World aggregate is present once per year and must be excluded from partner rankin
```

```
Validated 673 partner-total records across 3 years.
The World aggregate is present once per year and must be excluded from partner rankings.
```

3.4 Save the validated V2 partner-summary snapshot

In API mode, this cell saves the raw partner records, query scope, and request log in a new dated JSON file. It uses exclusive creation so it will not overwrite an existing snapshot, and stores no API key or request headers. Snapshot mode leaves the supplied file unchanged.

```
if DATA_SOURCE == "api":
    if len(extraction_log) != len(YEARS):
        raise ValueError("Complete all partner-summary requests before saving the snapshot."
    if sum(entry["records_received"] for entry in extraction_log) != len(list_trade):
        raise ValueError("Extraction log does not match the collected partner records.")

    saved_at = datetime.now(timezone.utc)
    snapshot_data = {
        "schema_version": 2,
        "source": "UN Comtrade",
        "source_url": "https://comtradeplus.un.org/ApiDocs",
        "saved_at_utc": saved_at.isoformat(),
        "query_scope": query_scope,
        "extraction_log": extraction_log,
        "records": list_trade
    }
    snapshot_folder = Path("data")
    snapshot_folder.mkdir(exist_ok=True)
    snapshot_name = f"v2_partner_totals_{saved_at:%Y%m%dT%H%M%S%fZ}.json"
    saved_snapshot_path = snapshot_folder / snapshot_name

    with saved_snapshot_path.open("x", encoding="utf-8") as file:
        json.dump(snapshot_data, file, ensure_ascii=False, indent=2, allow_nan=False)
```

```
    print(f"Saved {len(list_trade)} partner-total records to {saved_snapshot_path}.")
else:
    print("Using the supplied V2 snapshot; no new snapshot was written.")
```

```
Using the supplied V2 snapshot; no new snapshot was written.
```

```
# Use Pandas to inspect the records; keep list_trade as the raw MongoDB input.
preview_columns = [
    "period", "reporterDesc", "partnerCode", "partnerDesc",
    "cmdCode", "cmdDesc", "classificationCode", "primaryValue"
]
df_preview = pd.DataFrame(list_trade)
df_preview.reindex(columns=preview_columns).head(20)
```

	period	reporterDesc	partnerCode	partnerDesc	cmdCode	cmdDesc	classificati
0	2023	Switzerland	0	World	TOTAL	All Commodities	H6
1	2023	Switzerland	4	Afghanistan	TOTAL	All Commodities	H6
2	2023	Switzerland	8	Albania	TOTAL	All Commodities	H6
3	2023	Switzerland	12	Algeria	TOTAL	All Commodities	H6
4	2023	Switzerland	16	American Samoa	TOTAL	All Commodities	H6
5	2023	Switzerland	20	Andorra	TOTAL	All Commodities	H6
6	2023	Switzerland	24	Angola	TOTAL	All Commodities	H6
7	2023	Switzerland	28	Antigua and Barbuda	TOTAL	All Commodities	H6
8	2023	Switzerland	31	Azerbaijan	TOTAL	All Commodities	H6
9	2023	Switzerland	32	Argentina	TOTAL	All Commodities	H6
10	2023	Switzerland	36	Australia	TOTAL	All Commodities	H6
11	2023	Switzerland	40	Austria	TOTAL	All Commodities	H6
12	2023	Switzerland	44	Bahamas	TOTAL	All Commodities	H6
13	2023	Switzerland	48	Bahrain	TOTAL	All Commodities	H6
14	2023	Switzerland	50	Bangladesh	TOTAL	All Commodities	H6
15	2023	Switzerland	51	Armenia	TOTAL	All Commodities	H6
16	2023	Switzerland	52	Barbados	TOTAL	All Commodities	H6
17	2023	Switzerland	56	Belgium	TOTAL	All Commodities	H6
18	2023	Switzerland	60	Bermuda	TOTAL	All Commodities	H6
19	2023	Switzerland	64	Bhutan	TOTAL	All Commodities	H6

3.5 Understanding the V2 discovery data

3.5.1 What one record represents

V2's first extract contains one annual all-commodity export total for each reported partner area. It includes World as an aggregate row; it does not yet contain HS4 product detail. `partnerCode=None` omits the request filter; each returned record has its own `partnerCode` and `partnerDesc`. The code `TOTAL` combines all commodities for discovery.

The number of partner records is determined from each API response; do not assume every possible area has a reported record in every year. Keep the World total for reconciliation but exclude it, and any overlapping aggregate areas, when ranking individual partner areas.

3.5.2 Field guide: 47 source columns

Group	Columns	How to read them
Type and time	`typeCode`, `freqCode`, `period`, `refYear`, `refMonth`, `refPeriodId`	`C` = goods; `A` = annual. `period` is the year as text. `refMonth` can be `52` for annual data, not an ordinary month number.
Reporter	`reporterCode`, `reporterISO`, `reporterDesc`	Switzerland is `757`, `CHE`, `Switzerland`.
Flow	`flowCode`, `flowDesc`	`X`, `Export`.
Partners	`partnerCode`, `partnerISO`, `partnerDesc`, `partner2Code`, `partner2ISO`, `partner2Desc`	Partner fields vary by row; `partnerCode = 0` is World. The additional partner dimension remains aggregated.
Classification and commodity	`classificationCode`, `classificationSearchCode`, `isOriginalClassification`, `cmdCode`, `cmdDesc`, `aggrLevel`, `isLeaf`	The requested classification is `HS`; the returned edition must be `H6`. In this discovery query `cmdCode` is `TOTAL`, not an HS4 product code.
Customs and modes	`customsCode`, `customsDesc`, `mosCode`, `motCode`, `motDesc`	Source dimensions for customs and transport/mode selections; this query uses aggregate selections.
Quantities and weights	`qtyUnitCode`, `qtyUnitAbbr`, `qty`, `isQtyEstimated`, `altQtyUnitCode`, `altQtyUnitAbbr`, `altQty`, `isAltQtyEstimated`, `netWgt`, `isNetWgtEstimated`, `grossWgt`, `isGrossWgtEstimated`	Measures and units; estimation flags show when the measurement is estimated. Do not assume a missing or estimated zero is a measured zero.
Trade values	`cifvalue`, `fobvalue`, `primaryValue`	`primaryValue` is the main current-US-dollar measure for ranking. Do not add these valuation fields together.
Status and provenance	`legacyEstimationFlag`, `isReported`, `isAggregate`	Source-status and estimation information; check Comtrade definitions before interpreting a code.

3.5.3 Later HS4 detail and MongoDB model

After the top 30 partner areas are selected in MongoDB, V2 will request their H6 HS4 records. A clean document will represent one **year × partner area × HS4 heading** observation. Reporter, partner, commodity, trade, and quality fields can be embedded together. The raw partner summary and later raw HS4 data should remain unchanged; the clean top-50 subset will be produced in MongoDB after ranking.

The HS hierarchy is chapter (2 digits) → heading (4 digits) → subheading (6 digits). The discovery query contains `TOTAL`, so it cannot itself answer product-level questions. Do not combine parent totals with their detailed child codes in one sum.

3.5.4 HScode hierarchy example

`TOTAL` combines all commodities; it is not a numbered HS category. This example shows one branch, not product detail from the current dataset.

```
flowchart TD
    TOTAL["TOTAL: all commodities combined"]
    TOTAL --> SECTION["Section I: Live animals and animal products"]
    SECTION --> CHAPTER["01: Live animals"]
    CHAPTER --> HEADING["0101: Live horses, asses, mules and hinnies"]
    HEADING --> SUB1["010121: Purebred breeding horses"]
    HEADING --> SUB2["010129: Other horses"]
```

`H6` identifies HS 2022; HS4 identifies the four-digit heading level.

3.5.5 Future clean-record shape

After HS4 extraction and raw loading, a MongoDB transformation can create one document per year, partner, and HS4 heading. The raw collection will remain unchanged.

This example shows a possible clean model, not an existing MongoDB record. Germany and heading `0101` demonstrate the model, not confirmed top-30 or top-50 selections. The trade figures are illustrative, not downloaded HS4 results. The estimated-weight flag means we should not interpret `0.0` as a measured zero.


```
{
  "period": "2023",
  "reporter": {
    "code": 757,
    "iso": "CHE",
    "name": "Switzerland"
  },
  "partner": {
    "code": 276,
    "iso": "DEU",
    "name": "Germany"
  },
  "flow": {
    "code": "X",
    "description": "Export"
  },
  "commodity": {
    "edition": "H6",
    "level": 4,
    "code": "0101",
    "description": "Live horses, asses, mules and hinnies",
```

```
    "chapter": {
      "code": "01",
      "description": "Live animals"
    }
  },
  "trade": {
    "primaryValue": 12345.67,
    "fobValue": 12345.67,
    "netWeight": 0.0
  },
  "quality": {
    "netWeightEstimated": true
  }
}
```

```
flowchart TD
    RECORD["One clean observation"]
    RECORD --> YEAR["period: 2023"]
    RECORD --> REPORTER["reporter object: Switzerland"]
    RECORD --> PARTNER["partner object: Germany"]
    RECORD --> FLOW["flow object: Export"]
    RECORD --> COMMODITY["commodity object: HS4 heading 0101"]
    COMMODITY --> CHAPTER["chapter object: 01, Live animals"]
    RECORD --> TRADE["trade object: values and weight"]
    RECORD --> QUALITY["quality object: estimation flags"]
```

- Unlike the current flat records, this model stores related fields inside actual nested objects.
- Embedding groups fields inside one observation; it does not place partner observations inside the World record.
- `commodity.chapter` stores parent metadata, not another trade observation or an additional value to sum.
- Derive the parent chapter code from the heading prefix; obtain its description from an H6 reference table.
- HS4 extraction must supply actual product-level values; we cannot split current `TOTAL` values into products.
- Keep commodity codes as strings to preserve leading zeros.

For example, `record["partner"]["name"]` reads `Germany` in Python; MongoDB uses the field path `partner.name`.

3.6 Readable V2 partner-summary CSV

This CSV contains the partner discovery records and all source columns. It does not contain HS4 detail; that will be requested after the MongoDB stage selects the top 30 partners. Keep the JSON snapshot for reproducibility and load raw API records directly into MongoDB later, because CSV can lose types and leading zeros.

```
Path("data").mkdir(exist_ok=True)
trade_csv = Path("data") / f"v2_partner_totals_{datetime.now(timezone.utc):%Y%m%dT%H%M%S%fZ}
df_preview.to_csv(trade_csv, index=False, mode="x")
print(f"All {len(df_preview)} V2 partner-total records saved to {trade_csv}")
```

```
All 673 V2 partner-total records saved to data/v2_partner_totals_20261017T092929891101Z.csv
```

```
pd.DataFrame(extraction_log)[
    ["year_requested", "api_count", "records_received", "extracted_at_utc"]
]
```

	year_requested	api_count	records_received	extracted_at_utc
0	2023	226	226	2026-10-16T14:35:47.820549+00:00
1	2024	222	222	2026-10-16T14:35:54.531175+00:00
2	2025	225	225	2026-10-16T14:36:00.025211+00:00

3.7 Database setup: connect to Atlas

The course uses `pymongo.MongoClient(CNX_STR)` to connect to MongoDB. We keep that pattern but read `MONGODB_URI` from `.env` or the environment instead of notebook source. Keep the URI private and preserve the existing Comtrade key in `.env`.

Run the imports cell, then set `CONNECT_TO_MONGODB = True` below to enable this check. You do not need to rerun the API extraction. The default `False` skips credentials and network access, including during snapshot runs.

`MongoClient` creates a client; `client.admin.command("ping")` verifies connectivity. `serverSelectionTimeoutMS=10000` limits server-selection waiting to ten seconds. These checks extend the course connection example. They do not insert, update, or delete data.

Course reference: MongoDB Backends and Atlas.

```
%pip install -r requirements.txt
CONNECT_TO_MONGODB = True
```

```
Requirement already satisfied: requests<3,>=2.32.4 in /Users/dongyuangao/Desktop/mongodb_nos
packages (from -r requirements.txt (line 1)) (2.34.2)
Requirement already satisfied: pandas<4,>=2.2.3 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from -r requirements.txt (line 2)) (3.0.5)
Requirement already satisfied: python-dotenv==1.1.1 in /Users/dongyuangao/Desktop/mongodb_no
packages (from -r requirements.txt (line 3)) (1.1.1)
Requirement already satisfied: pymongo<5,>=4.6 in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from -r requirements.txt (line 4)) (4.18.1)
Requirement already satisfied: dnspython<3,>=2.6 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from -r requirements.txt (line 5)) (2.8.0)
Requirement already satisfied: jupyter<2,>=1.1.1 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from -r requirements.txt (line 6)) (1.1.1)
Requirement already satisfied: matplotlib<4,>=3.8 in /Users/dongyuangao/Desktop/mongodb_nosq
packages (from -r requirements.txt (line 7)) (3.11.2)
Requirement already satisfied: openpyxl==3.1.5 in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from -r requirements.txt (line 8)) (3.1.5)
Requirement already satisfied: et-xmlfile in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/
packages (from openpyxl==3.1.5->-r requirements.txt (line 8)) (2.0.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /Users/dongyuangao/Desktop/mongod
packages (from requests<3,>=2.32.4->-r requirements.txt (line 1)) (3.5.1)
```

```
Requirement already satisfied: idna<4,>=2.5 in /Users/dongyuangao/Desktop/mongodb_nosql/.ven
packages (from requests<3,>=2.32.4->-r requirements.txt (line 1)) (3.19)
Requirement already satisfied: urllib3<3,>=1.26 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from requests<3,>=2.32.4->-r requirements.txt (line 1)) (2.8.0)
Requirement already satisfied: certifi>=2023.5.7 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from requests<3,>=2.32.4->-r requirements.txt (line 1)) (2026.7.22)
Requirement already satisfied: numpy>=1.26.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.ve
packages (from pandas<4,>=2.2.3->-r requirements.txt (line 2)) (2.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/dongyuangao/Desktop/mongodb_
packages (from pandas<4,>=2.2.3->-r requirements.txt (line 2)) (2.9.0.post0)
Requirement already satisfied: notebook in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/li
packages (from jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (7.6.2)
Requirement already satisfied: jupyter-console in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (6.6.3)
Requirement already satisfied: nbconvert in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/l
packages (from jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (7.17.1)
Requirement already satisfied: ipykernel in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/l
packages (from jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (7.3.0)
Requirement already satisfied: ipywidgets in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/
packages (from jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (8.1.9)
Requirement already satisfied: jupyterlab in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/
packages (from jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (4.6.3)
Requirement already satisfied: contourpy>=1.0.1 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from matplotlib<4,>=3.8->-r requirements.txt (line 7)) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /Users/dongyuangao/Desktop/mongodb_nosql/.ven
packages (from matplotlib<4,>=3.8->-r requirements.txt (line 7)) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from matplotlib<4,>=3.8->-r requirements.txt (line 7)) (4.65.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from matplotlib<4,>=3.8->-r requirements.txt (line 7)) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from matplotlib<4,>=3.8->-r requirements.txt (line 7)) (26.3)
Requirement already satisfied: pillow>=9 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/l
packages (from matplotlib<4,>=3.8->-r requirements.txt (line 7)) (12.3.0)
Requirement already satisfied: pyparsing>=3 in /Users/dongyuangao/Desktop/mongodb_nosql/.ven
packages (from matplotlib<4,>=3.8->-r requirements.txt (line 7)) (3.3.2)
Requirement already satisfied: six>=1.5 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/li
packages (from python-dateutil>=2.8.2->pandas<4,>=2.2.3->-r requirements.txt (line 2)) (1.17
Requirement already satisfied: appnope>=0.1.2 in /Users/dongyuangao/Desktop/mongodb_nosql/.v
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (1.0.0)
Requirement already satisfied: comm>=0.1.1 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (0.2.3)
Requirement already satisfied: debugpy>=1.6.5 in /Users/dongyuangao/Desktop/mongodb_nosql/.v
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (1.8.22)
Requirement already satisfied: ipython>=7.23.1 in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (9.17.1)
Requirement already satisfied: jupyter-client>=8.9.0 in /Users/dongyuangao/Desktop/mongodb_n
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (8.10.0)
Requirement already satisfied: jupyter-core!=6.0.*,>=5.1 in /Users/dongyuangao/Desktop/mongo
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (5.9.1)
Requirement already satisfied: matplotlib-inline>=0.1 in /Users/dongyuangao/Desktop/mongodb_
```

```
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (0.2.2)
Requirement already satisfied: nest-asyncio2>=1.7.0 in /Users/dongyuangao/Desktop/mongodb_no
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (1.7.2)
Requirement already satisfied: psutil>=5.7 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (7.2.2)
Requirement already satisfied: pyzmq>=25 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/l
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (27.2.0)
Requirement already satisfied: tornado>=6.4.1 in /Users/dongyuangao/Desktop/mongodb_nosql/.v
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (6.5.10)
Requirement already satisfied: traitlets>=5.4.0 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (5.16.1)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in /Users/dongyuangao/Desktop/
packages (from ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6))
Requirement already satisfied: jedi>=0.18.2 in /Users/dongyuangao/Desktop/mongodb_nosql/.ven
packages (from ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6))
Requirement already satisfied: pexpect>4.6 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv
packages (from ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6))
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in /Users/dongyuangao/Desktop/m
packages (from ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6))
Requirement already satisfied: pygments>=2.14.0 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6))
Requirement already satisfied: stack_data>=0.6.0 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6))
Requirement already satisfied: typing_extensions>=4.6 in /Users/dongyuangao/Desktop/mongodb_
packages (from ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-r requirements.txt (line 6))
Requirement already satisfied: wcwidth>=0.1.4 in /Users/dongyuangao/Desktop/mongodb_nosql/.v
packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1-
r requirements.txt (line 6)) (0.8.3)
Requirement already satisfied: parso<0.9.0,>=0.8.6 in /Users/dongyuangao/Desktop/mongodb_nos
packages (from jedi>=0.18.2->ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (0.8.7)
Requirement already satisfied: platformdirs>=2.5 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from jupyter-core!=6.0.*,>=5.1->ipykernel->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (4.11.9)
Requirement already satisfied: ptyprocess>=0.5 in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from pexpect>4.6->ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (0.7.0)
Requirement already satisfied: executing>=1.2.0 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from stack_data>=0.6.0->ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from stack_data>=0.6.0->ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (3.0.2)
Requirement already satisfied: pure-eval in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/l
packages (from stack_data>=0.6.0->ipython>=7.23.1->ipykernel->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (0.2.4)
Requirement already satisfied: widgetsnbextension~=4.0.16 in /Users/dongyuangao/Desktop/mong
packages (from ipywidgets->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (4.0.16)
Requirement already satisfied: jupyterlab_widgets~=3.0.17 in /Users/dongyuangao/Desktop/mong
packages (from ipywidgets->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (3.0.17)
Requirement already satisfied: async-lru>=1.0.0 in /Users/dongyuangao/Desktop/mongodb_nosql/
```

```
packages (from jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (2.3.0)
Requirement already satisfied: httpx<1,>=0.25.0 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (0.28.1)
Requirement already satisfied: jinja2>=3.0.3 in /Users/dongyuangao/Desktop/mongodb_nosql/.ve
packages (from jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (3.1.6)
Requirement already satisfied: jupyter-builder>=1.0.2 in /Users/dongyuangao/Desktop/mongodb_
packages (from jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (1.2.3)
Requirement already satisfied: jupyter-lsp>=2.0.0 in /Users/dongyuangao/Desktop/mongodb_nosq
packages (from jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (2.3.1)
Requirement already satisfied: jupyter-server<3,>=2.19.0 in /Users/dongyuangao/Desktop/mongo
packages (from jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (2.21.1)
Requirement already satisfied: jupyterlab-server<3,>=2.28.0 in /Users/dongyuangao/Desktop/mo
packages (from jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (2.28.1)
Requirement already satisfied: notebook-shim>=0.2 in /Users/dongyuangao/Desktop/mongodb_nosq
packages (from jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (0.2.4)
Requirement already satisfied: anyio in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/lib/p
packages (from httpx<1,>=0.25.0->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)
Requirement already satisfied: httpcore==1.* in /Users/dongyuangao/Desktop/mongodb_nosql/.ve
packages (from httpx<1,>=0.25.0->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)
Requirement already satisfied: h11>=0.16 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/l
packages (from httpcore==1.*->httpx<1,>=0.25.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (0.16.0)
Requirement already satisfied: argon2-cffi>=21.1 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (25.1.0)
Requirement already satisfied: jupyter-events>=0.11.0 in /Users/dongyuangao/Desktop/mongodb_
packages (from jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (0.12.1)
Requirement already satisfied: jupyter-server-terminals>=0.4.4 in /Users/dongyuangao/Desktop
packages (from jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (0.5.4)
Requirement already satisfied: nbformat>=5.3.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (5.11.1)
Requirement already satisfied: overrides>=5.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.v
packages (from jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (7.7.0)
Requirement already satisfied: prometheus-client>=0.9 in /Users/dongyuangao/Desktop/mongodb_
packages (from jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (0.26.0)
Requirement already satisfied: send2trash>=1.8.2 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (2.1.0)
Requirement already satisfied: terminado>=0.8.3 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (0.18.1)
Requirement already satisfied: websocket-client>=1.7 in /Users/dongyuangao/Desktop/mongodb_n
packages (from jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (1.9.2)
Requirement already satisfied: babel>=2.10 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv
packages (from jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter<2,>=1.1.1->-
```

```
r requirements.txt (line 6)) (2.18.0)
Requirement already satisfied: json5>=0.9.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.ver
packages (from jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (0.15.0)
Requirement already satisfied: jsonschema>=4.18.0 in /Users/dongyuangao/Desktop/mongodb_nosq
packages (from jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (4.26.0)
Requirement already satisfied: argon2-cffi-bindings in /Users/dongyuangao/Desktop/mongodb_no
packages (from argon2-cffi>=21.1->jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->
r requirements.txt (line 6)) (26.1.0)
Requirement already satisfied: MarkupSafe>=2.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from jinja2>=3.0.3->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (
Requirement already satisfied: attrs>=22.2.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.ve
packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter<2,>=1.1
r requirements.txt (line 6)) (26.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /Users/dongyuangao/De
packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter<2,>=1.1
r requirements.txt (line 6)) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in /Users/dongyuangao/Desktop/mongodb_nos
packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter<2,>=1.1
r requirements.txt (line 6)) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from jsonschema>=4.18.0->jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter<2,>=1.1
r requirements.txt (line 6)) (2026.6.3)
Requirement already satisfied: python-json-logger>=2.0.4 in /Users/dongyuangao/Desktop/mongo
packages (from jupyter-events>=0.11.0->jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.
r requirements.txt (line 6)) (4.2.0)
Requirement already satisfied: pyyaml>=5.3 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv
packages (from jupyter-events>=0.11.0->jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.
r requirements.txt (line 6)) (6.0.3)
Requirement already satisfied: rfc3339-validator in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from jupyter-events>=0.11.0->jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.
r requirements.txt (line 6)) (0.1.4)
Requirement already satisfied: rfc3986-validator>=0.1.1 in /Users/dongyuangao/Desktop/mongod
packages (from jupyter-events>=0.11.0->jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.
r requirements.txt (line 6)) (0.1.1)
Requirement already satisfied: fqdn in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/lib/py
packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (1.5.1)
Requirement already satisfied: isoduration in /Users/dongyuangao/Desktop/mongodb_nosql/.venv
packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (20.11.0)
Requirement already satisfied: jsonpointer>1.13 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (3.1.1)
Requirement already satisfied: rfc3987-syntax>=1.1.0 in /Users/dongyuangao/Desktop/mongodb_r
packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (1.1.0)
Requirement already satisfied: uri-template in /Users/dongyuangao/Desktop/mongodb_nosql/.ver
packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (1.3.0)
```

```
Requirement already satisfied: webcolors>=24.6.0 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-
server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (25.10.0)
Requirement already satisfied: beautifulsoup4 in /Users/dongyuangao/Desktop/mongodb_nosql/.v
packages (from nbconvert->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (4.15.0)
Requirement already satisfied: bleach!=5.0.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.ve
packages (from bleach[css]!=5.0.0->nbconvert->jupyter<2,>=1.1.1->-r requirements.txt (line 6
Requirement already satisfied: defusedxml in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/
packages (from nbconvert->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (0.7.1)
Requirement already satisfied: jupyterlab-pygments in /Users/dongyuangao/Desktop/mongodb_nos
packages (from nbconvert->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (0.3.0)
Requirement already satisfied: mistune<4,>=2.0.3 in /Users/dongyuangao/Desktop/mongodb_nosql
packages (from nbconvert->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (3.3.4)
Requirement already satisfied: nbclient>=0.5.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from nbconvert->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (0.11.0)
Requirement already satisfied: pandocfilters>=1.4.1 in /Users/dongyuangao/Desktop/mongodb_no
packages (from nbconvert->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (1.5.1)
Requirement already satisfied: webencodings in /Users/dongyuangao/Desktop/mongodb_nosql/.ven
packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (0.6.1)
Requirement already satisfied: tinycss2>=1.1.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.
packages (from bleach[css]!=5.0.0->nbconvert->jupyter<2,>=1.1.1->-r requirements.txt (line 6
Requirement already satisfied: fastjsonschema>=2.15 in /Users/dongyuangao/Desktop/mongodb_no
packages (from nbformat>=5.3.0->jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (2.22.2)
Requirement already satisfied: lark>=1.2.2 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv
packages (from rfc3987-syntax>=1.1.0->jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.11.0->jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (1.3.1)
Requirement already satisfied: cffi>=1.0.1 in /Users/dongyuangao/Desktop/mongodb_nosql/.venv
packages (from argon2-cffi-bindings->argon2-cffi>=21.1->jupyter-server<3,>=2.19.0->jupyterla
r requirements.txt (line 6)) (2.1.1)
Requirement already satisfied: pycparser in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/l
packages (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi>=21.1->jupyter-
server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (3.0)
Requirement already satisfied: soupsieve>=1.6.1 in /Users/dongyuangao/Desktop/mongodb_nosql/
packages (from beautifulsoup4->nbconvert->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (
Requirement already satisfied: arrow>=0.15.0 in /Users/dongyuangao/Desktop/mongodb_nosql/.ve
packages (from isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyt
server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-r requirements.txt (line 6)) (1.4.0)
Requirement already satisfied: tzdata in /Users/dongyuangao/Desktop/mongodb_nosql/.venv/lib/
packages (from arrow>=0.15.0->isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.11.0->jupyter-server<3,>=2.19.0->jupyterlab->jupyter<2,>=1.1.1->-
r requirements.txt (line 6)) (2026.4)
Note: you may need to restart the kernel to use updated packages.
```

```python
if CONNECT_TO_MONGODB:
    load_dotenv(dotenv_path=".env", override=False)
    CNX_STR = os.environ.get("MONGODB_URI", "").strip()
    if not CNX_STR:
        raise ValueError("Set MONGODB_URI in .env or your environment before connecting.")
```

```
    client = pymongo.MongoClient(CNX_STR, serverSelectionTimeoutMS=10000)
    client.admin.command("ping")
    print("MongoDB connection successful.")
else:
    print("MongoDB connection skipped. Set CONNECT_TO_MONGODB = True to test Atlas.")
```

```
MongoDB connection successful.
```

3.8 V2 discovery interpretation and next steps

The discovery records contain annual all-commodity totals by partner area for 2023–2025. They are not the final HS4 dataset. Preserve this snapshot, then, after the MongoDB phase is authorized, load it into a raw collection and use an aggregation pipeline to rank partner areas by cumulative `primaryValue`.

Next, use the selected top-30 partner codes in a bounded H6 HS4 query. After loading those raw records, rank HS4 codes by value in MongoDB and create a clean top-50 collection. Keep World separate from partner rankings, do not fill missing combinations with zero, and document estimated or missing weights. V2 still needs the actual MongoDB model, analyses, diagrams, conclusions, and final PDF.

3.8.1 Planned MongoDB workflow

This workflow describes future work, not completed MongoDB steps. Rank partners across 2023-2025, then rank headings across those selected partners and years.

```
flowchart TD
    A["Current partner-total snapshot"]
    A --> B["Connect to MongoDB Atlas and verify connection"]
    B --> C["Load partner_totals_raw"]
    C --> D["MongoDB: rank top 30 eligible partners"]
    D --> E["Download HS4 detail for those partners"]
    E --> F["Load hs4_raw"]
    F --> G["MongoDB: rank top 50 HS4 headings"]
    G --> H["Transform selected observations into trade_clean"]
    H --> I["Aggregation analysis and charts"]
```

3.8.2 Select the database and collection

The course uses `db = client['db_test']` to select a database and `col = db.cl_test` to select a collection. We follow the same PyMongo pattern: `db` refers to `swiss_trade_v2`, and `collection_raw` refers to `partner_totals_raw`, where the original partner-total records will be stored before transformation.

These lines select database and collection handles; they do not insert or change documents. MongoDB creates them when the first documents are inserted. A count of zero means no documents were found yet.

```
DB_NAME = "swiss_trade_v2"
RAW_COLLECTION = "partner_totals_raw"

db = client[DB_NAME]
collection_raw = db[RAW_COLLECTION]

print("Database:", db.name)
print("Collection:", collection_raw.name)
print("Current documents:", collection_raw.count_documents({}))
```

```
Database: swiss_trade_v2
Collection: partner_totals_raw
Current documents: 673
```

```
# This code writes to Atlas.
if not list_trade:
    raise ValueError("Load and validate the snapshot first.")

for record in list_trade:
    document = record.copy()

    # Other query dimensions remain fixed in this collection.
    document["_id"] = f"{record['period']}_{record['partnerCode']}"

    collection_raw.replace_one(
        {"_id": document["_id"]},
        document,
        upsert=True
    )

print("Raw loading finished.")
print("Stored documents:", collection_raw.count_documents({}))
```

```
Raw loading finished.
Stored documents: 673
```

```
# Verify the stored count
for year in YEARS:
    count = collection_raw.count_documents({"period": year})
    print(year, count)

world_count = collection_raw.count_documents({"partnerCode": 0})
print("World records:", world_count)
```

```
2023 226
2024 222
2025 225
World records: 3
```

```
# Use MongoDB's aggregation pipeline, following the course's aggregate([...]) pattern.
# Sum three years of partner totals in MongoDB before requesting detailed HS4 data.
pipeline = [
    {
        # Keep Swiss export totals for the selected years; exclude World from partner rankir
        "$match": {
            "period": {"$in": YEARS},
            "reporterCode": 757,
            "flowCode": "X",
            "cmdCode": "TOTAL",
            "partnerCode": {"$ne": 0}
        }
    },
    {
        # Group by partner, sum the yearly values, and count that partner's records.
        "$group": {
            "_id": "$partnerCode",
            "partnerName": {"$first": "$partnerDesc"},
            "totalExportValue": {"$sum": "$primaryValue"},
            "observations": {"$sum": 1}
        }
    },
    {
        # Sort largest totals first; partner code gives ties a consistent order.
        "$sort": {
            "totalExportValue": -1,
            "_id": 1
        }
    },
    {
        # Keep only the top 30 partners for the later HS4 extraction.
        "$limit": 30
    }
]

# MongoDB runs the pipeline; Python receives only the small ranked result.
top30_candidates = list(collection_raw.aggregate(pipeline))

pd.DataFrame(top30_candidates).rename(
    columns={"_id": "partnerCode"}
)
top30_table = pd.DataFrame(top30_candidates).rename(
    columns={"_id": "partnerCode"}
)

print(top30_table.to_string(index=False))
```

```
 partnerCode          partnerName  totalExportValue  observations
         842                  USA      2.571219e+11             3
         276              Germany      1.625739e+11             3
         156                China      1.229667e+11             3
```

826	United Kingdom	9.536637e+10	3
380	Italy	8.390953e+10	3
705	Slovenia	7.879933e+10	3
251	France	7.345312e+10	3
699	India	5.535842e+10	3
792	Türkiye	3.841508e+10	3
344	China, Hong Kong SAR	3.507132e+10	3
784	United Arab Emirates	3.380909e+10	3
40	Austria	3.101915e+10	3
392	Japan	2.737813e+10	3
724	Spain	2.333638e+10	3
702	Singapore	2.215341e+10	3
528	Netherlands	2.205905e+10	3
682	Saudi Arabia	2.067366e+10	3
56	Belgium	1.699624e+10	3
764	Thailand	1.534239e+10	3
124	Canada	1.420964e+10	3
410	Rep. of Korea	1.197956e+10	3
616	Poland	1.092910e+10	3
76	Brazil	1.025735e+10	3
490	Other Asia, nes	9.003020e+09	3
36	Australia	8.567268e+09	3
643	Russian Federation	8.072643e+09	3
484	Mexico	7.423717e+09	3
203	Czechia	7.259918e+09	3
642	Romania	6.641414e+09	3
752	Sweden	6.324772e+09	3

```
selected_partner_codes = [
    row["_id"] for row in top30_candidates
]

if len(selected_partner_codes) != 30:
    raise ValueError("Expected 30 selected partners.")

if len(set(selected_partner_codes)) != 30:
    raise ValueError("Partner codes must be unique.")

if 0 in selected_partner_codes:
    raise ValueError("World must not appear in the partner selection.")

partner_codes_parameter = ",".join(
    str(code) for code in selected_partner_codes
)

print(partner_codes_parameter)
```

```
842,276,156,826,380,705,251,699,792,344,784,40,392,724,702,528,682,56,764,124,410,616,76,490
```

3.9 Top 30 partner countries chosen from above

The scope includes the top 30 non-World partner areas by cumulative 2023-2025 export value. We retain code `490`, `Other Asia, nes`, with its official source label. The scope therefore covers partner areas, not only sovereign countries.

The HS4 snapshot will preserve the selected codes and ranking results. We will use the same partner selection for every year.

3.10 Extract HS4 detail for the selected partners

`AG4` requests four-digit HS headings; it does not mean the top four products. We need all returned headings before MongoDB can rank the top 50. The response must use original H6 classification and aggregation level 4.

Keep the discovery `DATA_SOURCE` in snapshot mode. HS4 uses a separate `HS4_DATA_SOURCE`: `off`, `api`, or `snapshot`. The default `off` performs no extraction. Only `api` loads the Comtrade key and sends requests; `snapshot` reads a saved HS4 file without credentials or network access.

We request one partner and year per batch: 90 data requests before retries, plus one public H6 reference download. Successful data batches have at least five seconds between them. Check your remaining quota before starting. The shared helper checks response counts without truncating fractional values.

Run the updated download-function cell before this section. Never print request headers or credentials. An empty batch means the API returned no rows; it does not establish zero exports.

```
HS4_DATA_SOURCE = "api"  # off, api, or snapshot
HS4_SNAPSHOT_PATH = Path("data/v2_hs4_snapshot.json")
HS4_RAW_COLLECTION = "hs4_raw"
HS4_REFERENCE_URL = "https://comtradeapi.un.org/files/v1/app/reference/H6.json"
LOAD_HS4_TO_MONGODB = True
RANK_HS4_IN_MONGODB = True

hs4_base_params = base_params.copy()
hs4_base_params["cmdCode"] = "AG4"
hs4_base_params.pop("partnerCode", None)

if HS4_DATA_SOURCE not in {"off", "api", "snapshot"}:
    raise ValueError("Choose off, api, or snapshot for HS4_DATA_SOURCE.")
```

```
def make_hs4_scope(partner_codes):
    if not isinstance(partner_codes, list) or len(partner_codes) != 30:
        raise ValueError("HS4 extraction requires 30 selected partner codes.")
    if any(isinstance(code, bool) or not isinstance(code, int) or code <= 0 for code in part
        raise ValueError("HS4 partner codes must be positive integers; exclude World.")
    if len(set(partner_codes)) != 30:
        raise ValueError("HS4 partner codes must be unique.")
    return {
        "endpoint": API_URL,
        "years": YEARS.copy(),
        "filters": hs4_base_params.copy(),
```

```
        "expected_edition": EXPECTED_EDITION,
        "selected_partner_codes": partner_codes.copy()
    }


def validate_hs4_batch(records, year, partner_code):
    if not isinstance(records, list):
        raise ValueError("Expected a list of HS4 records.")
    expected = {
        "period": year, "partnerCode": partner_code,
        "reporterCode": hs4_base_params["reporterCode"],
        "typeCode": "C", "freqCode": "A", "aggrLevel": 4,
        "classificationCode": EXPECTED_EDITION, "isOriginalClassification": True,
        "flowCode": hs4_base_params["flowCode"],
        "partner2Code": hs4_base_params["partner2Code"],
        "customsCode": hs4_base_params["customsCode"],
        "motCode": hs4_base_params["motCode"]
    }
    codes = []
    for record in records:
        if not isinstance(record, dict):
            raise ValueError("Each HS4 record must be an object.")
        for field, value in expected.items():
            if record.get(field) != value:
                raise ValueError(f"Unexpected HS4 field {field} for {year}, partner {partner
        code = record.get("cmdCode")
        if not isinstance(code, str) or len(code) != 4 or not code.isascii() or not code.isd
            raise ValueError("Expected a four-digit HS4 code, not TOTAL or another level.")
        value = record.get("primaryValue")
        if isinstance(value, bool) or not isinstance(value, (int, float)):
            raise ValueError("Each HS4 record needs a numeric primaryValue.")
        if not math.isfinite(value) or value < 0:
            raise ValueError("HS4 primaryValue must be finite and nonnegative.")
        codes.append(code)
    if len(codes) != len(set(codes)):
        raise ValueError("Duplicate year/partner/HS4 records found.")


def validate_hs4_extract(records, extraction_log, scope):
    if not isinstance(scope, dict) or scope != make_hs4_scope(scope.get("selected_partner_co
        raise ValueError("HS4 snapshot query scope differs from the notebook settings.")
    if not isinstance(records, list) or not records:
        raise ValueError("No HS4 records are available.")
    if not isinstance(extraction_log, list):
        raise ValueError("Expected an HS4 extraction log.")
    batches = {
        (year, partner): []
        for year in scope["years"] for partner in scope["selected_partner_codes"]
    }
    for record in records:
        if not isinstance(record, dict):
```

```
            raise ValueError("Each HS4 record must be an object.")
        key = (record.get("period"), record.get("partnerCode"))
        if key not in batches:
            raise ValueError("HS4 record falls outside the selected years or partners.")
        batches[key].append(record)
    seen = set()
    for entry in extraction_log:
        if not isinstance(entry, dict):
            raise ValueError("Each HS4 extraction-log entry must be an object.")
        key = (entry.get("year_requested"), entry.get("partner_requested"))
        if key not in batches or key in seen:
            raise ValueError("HS4 extraction log has an unexpected or repeated batch.")
        seen.add(key)
        count = entry.get("api_count")
        if isinstance(count, bool) or not isinstance(count, int) or not 0 <= count < MAX_REC
            raise ValueError("Invalid HS4 response count or record cap reached.")
        if count != entry.get("records_received") or count != len(batches[key]):
            raise ValueError("HS4 batch counts do not match the records.")
        expected_query = {**hs4_base_params, "period": key[0], "partnerCode": key[1]}
        if entry.get("query") != expected_query or entry.get("endpoint") != API_URL:
            raise ValueError("HS4 batch query does not match the requested scope.")
        validate_hs4_batch(batches[key], key[0], key[1])
    if seen != set(batches):
        raise ValueError("HS4 extraction log must cover all 90 year/partner batches.")
```

3.10.1 Reference codes and value coverage

The H6 reference identifies standard headings; four digits alone do not prove that a code is a valid heading. API mode downloads this public reference once. The HS4 snapshot preserves it, so later snapshot runs need no reference download.

We preserve nonstandard source codes in the raw data and list them separately. The top-50 pipeline includes only standard H6 headings. Reference chapters 00, 77, 98, and 99 do not represent standard international HS chapters.

A Pandas quality check compares each partner-year's HS4 value sum with its frozen all-commodity total. It also reports coverage from standard headings alone. Differences need investigation; they do not justify automatic rescaling or inventing missing observations.

```
def get_hs4_heading_codes(reference):
    if not isinstance(reference, dict) or reference.get("source_url") != HS4_REFERENCE_URL:
        raise ValueError("Expected the saved H6 reference and its source URL.")
    payload = reference.get("data")
    if not isinstance(payload, dict) or not isinstance(payload.get("results"), list):
        raise ValueError("Expected a results list in the H6 reference.")
    heading_codes = set()
    for item in payload["results"]:
        if not isinstance(item, dict):
            raise ValueError("Each H6 reference entry must be an object.")
        code = item.get("id")
        if isinstance(code, str) and len(code) == 4 and code.isascii() and code.isdigit():
```

```
            chapter = int(code[:2])
            if 1 <= chapter <= 97 and chapter != 77:
                heading_codes.add(code)
    if not heading_codes:
        raise ValueError("The H6 reference contains no standard four-digit headings.")
    return heading_codes


def validate_hs4_control_totals(control_totals, scope):
    if not isinstance(control_totals, list):
        raise ValueError("Expected saved partner-year control totals.")
    expected_keys = {
        (year, partner)
        for year in scope["years"] for partner in scope["selected_partner_codes"]
    }
    seen = set()
    for record in control_totals:
        if not isinstance(record, dict):
            raise ValueError("Each control total must be an object.")
        key = (record.get("period"), record.get("partnerCode"))
        if key not in expected_keys or key in seen:
            raise ValueError("Unexpected or duplicate partner-year control total.")
        value = record.get("primaryValue")
        if isinstance(value, bool) or not isinstance(value, (int, float)):
            raise ValueError("Control totals must contain numeric trade values.")
        if not math.isfinite(value) or value < 0:
            raise ValueError("Control totals must be finite and nonnegative.")
        seen.add(key)
    if seen != expected_keys:
        raise ValueError("Control totals must cover every selected partner and year.")


def hs4_coverage_table(records, control_totals, scope, heading_codes):
    validate_hs4_control_totals(control_totals, scope)
    keys = ["period", "partnerCode"]
    details = pd.DataFrame(records, columns=keys + ["cmdCode", "primaryValue"])
    totals = pd.DataFrame(control_totals)[keys + ["primaryValue"]]
    totals = totals.rename(columns={"primaryValue": "totalValue"})
    raw_sums = details.groupby(keys, as_index=False)["primaryValue"].sum()
    raw_sums = raw_sums.rename(columns={"primaryValue": "hs4Value"})
    standard_rows = details[details["cmdCode"].isin(heading_codes)]
    standard_sums = standard_rows.groupby(keys, as_index=False)["primaryValue"].sum()
    standard_sums = standard_sums.rename(columns={"primaryValue": "standardHeadingValue"})
    coverage = totals.merge(raw_sums, on=keys, how="left", validate="one_to_one")
    coverage = coverage.merge(standard_sums, on=keys, how="left", validate="one_to_one")
    coverage["differenceUSD"] = coverage["hs4Value"] - coverage["totalValue"]
    denominator = coverage["totalValue"].where(coverage["totalValue"] != 0)
    coverage["coveragePct"] = 100 * coverage["hs4Value"] / denominator
    coverage["standardCoveragePct"] = 100 * coverage["standardHeadingValue"] / denominator
    coverage["absoluteDifferenceUSD"] = coverage["differenceUSD"].abs()
    return coverage
```

3.10.2 Run extraction or reproduce a snapshot

For the first download, set `HS4_DATA_SOURCE = "api"` only in the settings cell. Keep the extraction cell's `if` and `elif` conditions unchanged. Run the settings, validation-functions, reference-and-coverage-functions, and extraction cells in order. The notebook loads the API key separately from the discovery mode. Existing partner totals and their settings remain unchanged.

For reproduction, set `HS4_DATA_SOURCE = "snapshot"` and provide the exact saved filename. Snapshot mode restores the frozen partner selection and checks it against the current years, filters, and edition. It does not rerun the partner ranking or require Atlas for these HS4 cells.

```
hs4_records = []
hs4_extraction_log = []
hs4_query_scope = None
hs4_partner_ranking = []
hs4_reference = None
hs4_control_totals = []
hs4_heading_codes = set()
hs4_nonstandard_codes = []

if HS4_DATA_SOURCE == "snapshot":
    with HS4_SNAPSHOT_PATH.open(encoding="utf-8") as file:
        hs4_snapshot = json.load(file)
    if not isinstance(hs4_snapshot, dict) or hs4_snapshot.get("dataset") != "swiss_exports_h
        raise ValueError("Expected an HS4 snapshot, not the partner-total snapshot.")
    if hs4_snapshot.get("schema_version") != 2:
        raise ValueError("Unsupported HS4 snapshot version.")
    hs4_query_scope = hs4_snapshot.get("query_scope")
    hs4_records = hs4_snapshot.get("records")
    hs4_extraction_log = hs4_snapshot.get("extraction_log")
    hs4_partner_ranking = hs4_snapshot.get("partner_ranking")
    hs4_reference = hs4_snapshot.get("heading_reference")
    hs4_control_totals = hs4_snapshot.get("control_totals")
elif HS4_DATA_SOURCE == "api":
    hs4_query_scope = make_hs4_scope(selected_partner_codes)
    hs4_partner_ranking = [row.copy() for row in top30_candidates]
    if [row["_id"] for row in hs4_partner_ranking] != selected_partner_codes:
        raise ValueError("Selected partner codes differ from the MongoDB ranking.")
    for record in list_trade:
        if record["period"] in YEARS and record["partnerCode"] in selected_partner_codes:
            hs4_control_totals.append({
                "period": record["period"], "partnerCode": record["partnerCode"],
                "primaryValue": record["primaryValue"]
            })
    validate_hs4_control_totals(hs4_control_totals, hs4_query_scope)
    load_dotenv(dotenv_path=".env", override=False)
    hs4_api_key = os.environ.get("UN_COMTRADE_API_KEY", "").strip()
    if not hs4_api_key:
        hs4_api_key = getpass("UN Comtrade API key for HS4 extraction: ").strip()
    if not hs4_api_key:
        raise ValueError("An API key is required for HS4 extraction.")
```

```
    hs4_headers = {"Ocp-Apim-Subscription-Key": hs4_api_key}
    reference_response = requests.get(HS4_REFERENCE_URL, timeout=45)
    reference_response.raise_for_status()
    hs4_reference = {
        "source_url": HS4_REFERENCE_URL,
        "downloaded_at_utc": datetime.now(timezone.utc).isoformat(),
        "data": reference_response.json()
    }
    get_hs4_heading_codes(hs4_reference)
    batch_number = 0
    batch_total = len(YEARS) * len(selected_partner_codes)
    for year in YEARS:
        for partner_code in selected_partner_codes:
            batch_number += 1
            print(f"HS4 batch {batch_number}/{batch_total}: {year}, partner {partner_code}")
            params = {**hs4_base_params, "partnerCode": partner_code}
            extracted_at = datetime.now(timezone.utc).isoformat()
            records, api_count = request_comtrade_records(year, params, hs4_headers)
            validate_hs4_batch(records, year, partner_code)
            hs4_records.extend(records)
            hs4_extraction_log.append({
                "year_requested": year, "partner_requested": partner_code,
                "api_count": api_count, "records_received": len(records),
                "extracted_at_utc": extracted_at, "endpoint": API_URL,
                "query": {**params, "period": year}
            })
            if batch_number < batch_total:
                time.sleep(max(5, REQUEST_DELAY))
else:
    print("HS4 extraction skipped. Choose api or snapshot when ready.")

if HS4_DATA_SOURCE in {"api", "snapshot"}:
    validate_hs4_extract(hs4_records, hs4_extraction_log, hs4_query_scope)
    if not isinstance(hs4_partner_ranking, list) or any(not isinstance(row, dict) for row in
        raise ValueError("HS4 snapshot must preserve the partner ranking.")
    if [row.get("_id") for row in hs4_partner_ranking] != hs4_query_scope["selected_partner_
        raise ValueError("HS4 snapshot partner ranking differs from its selected codes.")
    hs4_heading_codes = get_hs4_heading_codes(hs4_reference)
    validate_hs4_control_totals(hs4_control_totals, hs4_query_scope)
    source_codes = {record["cmdCode"] for record in hs4_records}
    hs4_nonstandard_codes = sorted(source_codes - hs4_heading_codes)
    print(f"Validated {len(hs4_records)} HS4 records from {len(hs4_extraction_log)} batches.
    print("Nonstandard source codes:", hs4_nonstandard_codes)
```

```
HS4 batch 1/90: 2023, partner 842
HS4 batch 2/90: 2023, partner 276
HS4 batch 3/90: 2023, partner 156
HS4 batch 4/90: 2023, partner 826
HS4 batch 5/90: 2023, partner 380
HS4 batch 6/90: 2023, partner 705
HS4 batch 7/90: 2023, partner 251
```

```
HS4 batch 8/90: 2023, partner 699
HS4 batch 9/90: 2023, partner 792
HS4 batch 10/90: 2023, partner 344
HS4 batch 11/90: 2023, partner 784
HS4 batch 12/90: 2023, partner 40
HS4 batch 13/90: 2023, partner 392
HS4 batch 14/90: 2023, partner 724
HS4 batch 15/90: 2023, partner 702
HS4 batch 16/90: 2023, partner 528
HS4 batch 17/90: 2023, partner 682
HS4 batch 18/90: 2023, partner 56
HS4 batch 19/90: 2023, partner 764
HS4 batch 20/90: 2023, partner 124
HS4 batch 21/90: 2023, partner 410
HS4 batch 22/90: 2023, partner 616
HS4 batch 23/90: 2023, partner 76
HS4 batch 24/90: 2023, partner 490
HS4 batch 25/90: 2023, partner 36
HS4 batch 26/90: 2023, partner 643
HS4 batch 27/90: 2023, partner 484
HS4 batch 28/90: 2023, partner 203
HS4 batch 29/90: 2023, partner 642
HS4 batch 30/90: 2023, partner 752
HS4 batch 31/90: 2024, partner 842
HS4 batch 32/90: 2024, partner 276
HS4 batch 33/90: 2024, partner 156
HS4 batch 34/90: 2024, partner 826
HS4 batch 35/90: 2024, partner 380
HS4 batch 36/90: 2024, partner 705
HS4 batch 37/90: 2024, partner 251
HS4 batch 38/90: 2024, partner 699
HS4 batch 39/90: 2024, partner 792
HS4 batch 40/90: 2024, partner 344
HS4 batch 41/90: 2024, partner 784
HS4 batch 42/90: 2024, partner 40
HS4 batch 43/90: 2024, partner 392
HS4 batch 44/90: 2024, partner 724
HS4 batch 45/90: 2024, partner 702
HS4 batch 46/90: 2024, partner 528
HS4 batch 47/90: 2024, partner 682
HS4 batch 48/90: 2024, partner 56
HS4 batch 49/90: 2024, partner 764
HS4 batch 50/90: 2024, partner 124
HS4 batch 51/90: 2024, partner 410
HS4 batch 52/90: 2024, partner 616
HS4 batch 53/90: 2024, partner 76
HS4 batch 54/90: 2024, partner 490
HS4 batch 55/90: 2024, partner 36
HS4 batch 56/90: 2024, partner 643
HS4 batch 57/90: 2024, partner 484
HS4 batch 58/90: 2024, partner 203
```

```
HS4 batch 59/90: 2024, partner 642
HS4 batch 60/90: 2024, partner 752
HS4 batch 61/90: 2025, partner 842
HS4 batch 62/90: 2025, partner 276
HS4 batch 63/90: 2025, partner 156
HS4 batch 64/90: 2025, partner 826
HS4 batch 65/90: 2025, partner 380
HS4 batch 66/90: 2025, partner 705
HS4 batch 67/90: 2025, partner 251
HS4 batch 68/90: 2025, partner 699
HS4 batch 69/90: 2025, partner 792
HS4 batch 70/90: 2025, partner 344
HS4 batch 71/90: 2025, partner 784
HS4 batch 72/90: 2025, partner 40
HS4 batch 73/90: 2025, partner 392
HS4 batch 74/90: 2025, partner 724
HS4 batch 75/90: 2025, partner 702
HS4 batch 76/90: 2025, partner 528
HS4 batch 77/90: 2025, partner 682
HS4 batch 78/90: 2025, partner 56
HS4 batch 79/90: 2025, partner 764
HS4 batch 80/90: 2025, partner 124
HS4 batch 81/90: 2025, partner 410
HS4 batch 82/90: 2025, partner 616
HS4 batch 83/90: 2025, partner 76
HS4 batch 84/90: 2025, partner 490
HS4 batch 85/90: 2025, partner 36
HS4 batch 86/90: 2025, partner 643
HS4 batch 87/90: 2025, partner 484
HS4 batch 88/90: 2025, partner 203
HS4 batch 89/90: 2025, partner 642
HS4 batch 90/90: 2025, partner 752
Validated 72459 HS4 records from 90 batches.
Nonstandard source codes: ['9999']
```

3.10.3 Preserve the validated HS4 extraction

Save only after all 90 batches pass validation. Snapshot schema version 2 preserves raw records, logs, query settings, partner ranking, H6 reference, and partner-year control totals. It contains no request headers or API keys. Exclusive file creation prevents overwriting an existing snapshot.

Copy the printed filename into `HS4_SNAPSHOT_PATH` before switching to snapshot mode. Missing partner-product-year combinations remain absent; we do not manufacture zero-valued observations.

```
if HS4_DATA_SOURCE == "api":
    validate_hs4_extract(hs4_records, hs4_extraction_log, hs4_query_scope)
    get_hs4_heading_codes(hs4_reference)
    validate_hs4_control_totals(hs4_control_totals, hs4_query_scope)
    saved_at = datetime.now(timezone.utc)
    hs4_snapshot_data = {
```

```
        "schema_version": 2, "dataset": "swiss_exports_hs4",
        "source": "UN Comtrade", "source_url": "https://comtradeplus.un.org/ApiDocs",
        "saved_at_utc": saved_at.isoformat(),
        "query_scope": hs4_query_scope, "partner_ranking": hs4_partner_ranking,
        "heading_reference": hs4_reference, "control_totals": hs4_control_totals,
        "extraction_log": hs4_extraction_log, "records": hs4_records
    }
    Path("data").mkdir(exist_ok=True)
    hs4_saved_snapshot_path = Path("data") / f"v2_hs4_snapshot_{saved_at:%Y%m%dT%H%M%S%fZ}.j
    with hs4_saved_snapshot_path.open("x", encoding="utf-8") as file:
        json.dump(hs4_snapshot_data, file, ensure_ascii=False, indent=2, allow_nan=False)
    print(f"Saved {len(hs4_records)} HS4 records to {hs4_saved_snapshot_path}.")
elif HS4_DATA_SOURCE == "snapshot":
    print("Using the supplied HS4 snapshot; no new snapshot was written.")
else:
    print("HS4 snapshot saving skipped.")
```

```
Saved 72459 HS4 records to data/v2_hs4_snapshot_20261017T094752738553Z.json.
```

```
if HS4_DATA_SOURCE in {"api", "snapshot"}:
    hs4_preview = pd.DataFrame(hs4_records)
    print(hs4_preview[["period", "partnerCode", "partnerDesc", "cmdCode", "cmdDesc", "primar
    hs4_log_table = pd.DataFrame(hs4_extraction_log)
    print(hs4_log_table.groupby("year_requested")["records_received"].sum().to_string())
    print("Empty batches:", (hs4_log_table["records_received"] == 0).sum())
```

```
period  partnerCode partnerDesc cmdCode
  2023          842         USA    0101
  2023          842         USA    0106
  2023          842         USA    0210
  2023          842         USA    0301
  2023          842         USA    0402
  2023          842         USA    0403 Yogurt; buttermilk, curdled milk and cream, kephir a
  2023          842         USA    0404
  2023          842         USA    0405
  2023          842         USA    0406
  2023          842         USA    0409
year_requested
2023    23835
2024    24218
2025    24406
Empty batches: 0
```

3.10.4 Compare HS4 coverage with partner totals

The table compares all returned four-digit source rows and standard H6 headings with the discovery totals. `differenceUSD` means the source-row sum minus the all-commodity total. `coveragePct` and `standardCoveragePct` report the two coverage percentages.

Review the largest differences before interpreting rankings or shares. Missing HS4 batches retain missing coverage, not zero coverage. A zero control total has no defined coverage percentage. We do not rescale values or require exact equality, because source rounding and reporting coverage can differ.

```
if HS4_DATA_SOURCE in {"api", "snapshot"}:
    validate_hs4_extract(hs4_records, hs4_extraction_log, hs4_query_scope)
    hs4_heading_codes = get_hs4_heading_codes(hs4_reference)
    hs4_coverage = hs4_coverage_table(
        hs4_records, hs4_control_totals, hs4_query_scope, hs4_heading_codes
    )
    largest_gaps = hs4_coverage.sort_values("absoluteDifferenceUSD", ascending=False)
    print(largest_gaps.head(10).to_string(index=False))
    print("Partner-years without HS4 rows:", hs4_coverage["hs4Value"].isna().sum())
    hs4_nonstandard_codes = sorted({row["cmdCode"] for row in hs4_records} - hs4_heading_cod
    print("Nonstandard source codes excluded from heading ranking:", hs4_nonstandard_codes)
```

```
period  partnerCode   totalValue     hs4Value  standardHeadingValue  differenceUSD  coverage
  2024          764 4.954415e+09 4.954415e+09          4.949646e+09       0.018001        10
  2024          702 6.384549e+09 6.384549e+09          6.359975e+09       0.017000        10
  2023          642 1.805641e+09 1.805641e+09          1.801693e+09       0.016000        10
  2025          203 2.597759e+09 2.597759e+09          2.596291e+09       0.016000        10
  2024          203 2.331849e+09 2.331849e+09          2.330070e+09       0.015000        10
  2025          380 2.919904e+10 2.919904e+10          2.918047e+10       0.014999        10
  2024          826 2.223955e+10 2.223955e+10          2.222570e+10       0.014000        10
  2025          702 7.181781e+09 7.181781e+09          7.155719e+09       0.014000        10
  2024          682 7.235734e+09 7.235734e+09          7.231484e+09      -
0.013000        100.0            99.941257               0.013000
  2025           36 2.951541e+09 2.951541e+09          2.949316e+09       0.013000        10
Partner-years without HS4 rows: 0
Nonstandard source codes excluded from heading ranking: ['9999']
```

3.11 Load HS4 records and rank headings in MongoDB

Only enable `LOAD_HS4_TO_MONGODB` after saving the snapshot and reviewing coverage. The loader keeps the course's replacement/upsert approach. It uses PyMongo `ReplaceOne` operations with `bulk_write()` in batches of at most 1,000. This small extension reduces Atlas round trips without changing document contents or rerun behavior.

Reruns replace matching observations instead of duplicating them. The loader refuses stale observations absent from the current snapshot before writing. Use a new collection for incompatible snapshots; the notebook never deletes existing records automatically.

`RANK_HS4_IN_MONGODB` enables a separate read-only pipeline. The check first verifies that stored source fields match the current snapshot. `$match` includes only headings from the saved H6 reference; nonstandard source rows remain in `hs4_raw`. `$group`, `$sum`, `$sort`, and `$limit` rank 50 headings across all selected partners and years, not 50 per partner.

Course references: CRUD and aggregation examples. Pandas displays the returned results; MongoDB performs the ranking.

```
def hs4_document_id(record):
    fields = [
        "reporterCode", "period", "partnerCode", "partner2Code", "cmdCode",
        "classificationCode", "flowCode", "customsCode", "motCode"
    ]
    return "_".join(str(record[field]) for field in fields)


def hs4_mongodb_filter(scope):
    filters = scope["filters"]
    return {
        "period": {"$in": scope["years"]},
        "partnerCode": {"$in": scope["selected_partner_codes"]},
        "reporterCode": filters["reporterCode"],
        "flowCode": filters["flowCode"],
        "partner2Code": filters["partner2Code"],
        "customsCode": filters["customsCode"], "motCode": filters["motCode"],
        "typeCode": "C", "freqCode": "A", "aggrLevel": 4,
        "classificationCode": scope["expected_edition"],
        "isOriginalClassification": True,
        "cmdCode": {"$regex": "^[0-9]{4}$"}
    }


def verify_hs4_collection(collection, records, scope):
    expected = {hs4_document_id(record): record for record in records}
    seen = set()
    for document in collection.find(hs4_mongodb_filter(scope)):
        source_record = document.copy()
        document_id = source_record.pop("_id")
        if document_id not in expected or source_record != expected[document_id]:
            raise ValueError("Stored HS4 records differ from the snapshot; check the raw loa
        seen.add(document_id)
    if seen != set(expected):
        raise ValueError("Stored HS4 records are incomplete; run the HS4 load first.")
    return len(seen)


def load_hs4_records(collection, records, extraction_log, scope):
    validate_hs4_extract(records, extraction_log, scope)
    expected_ids = {hs4_document_id(record) for record in records}
    stored_ids = {
        document["_id"]
        for document in collection.find(hs4_mongodb_filter(scope), {"_id": 1})
    }
    if stored_ids - expected_ids:
        raise ValueError("Existing HS4 scope contains stale records. Use a new collection; r
    operations = []
    for record in records:
        document = record.copy()
        document["_id"] = hs4_document_id(record)
```

```
        operations.append(pymongo.ReplaceOne({"_id": document["_id"]}, document, upsert=True
        if len(operations) == 1000:
            collection.bulk_write(operations, ordered=False)
            operations = []
    if operations:
        collection.bulk_write(operations, ordered=False)
    return verify_hs4_collection(collection, records, scope)
```

```
if LOAD_HS4_TO_MONGODB:
    if not CONNECT_TO_MONGODB:
        raise ValueError("Enable and verify the Atlas connection before loading HS4 records.
    if HS4_RAW_COLLECTION == RAW_COLLECTION:
        raise ValueError("Use a separate collection for HS4 detail, not partner totals.")
    collection_hs4_raw = db[HS4_RAW_COLLECTION]
    hs4_stored_count = load_hs4_records(
        collection_hs4_raw, hs4_records, hs4_extraction_log, hs4_query_scope
    )
    print(f"Verified {hs4_stored_count} stored HS4 documents in {HS4_RAW_COLLECTION}.")
else:
    print("HS4 MongoDB loading skipped. Enable LOAD_HS4_TO_MONGODB when ready.")
```

```
Verified 72459 stored HS4 documents in hs4_raw.
```

3.11.1 HS4 raw documents in Atlas

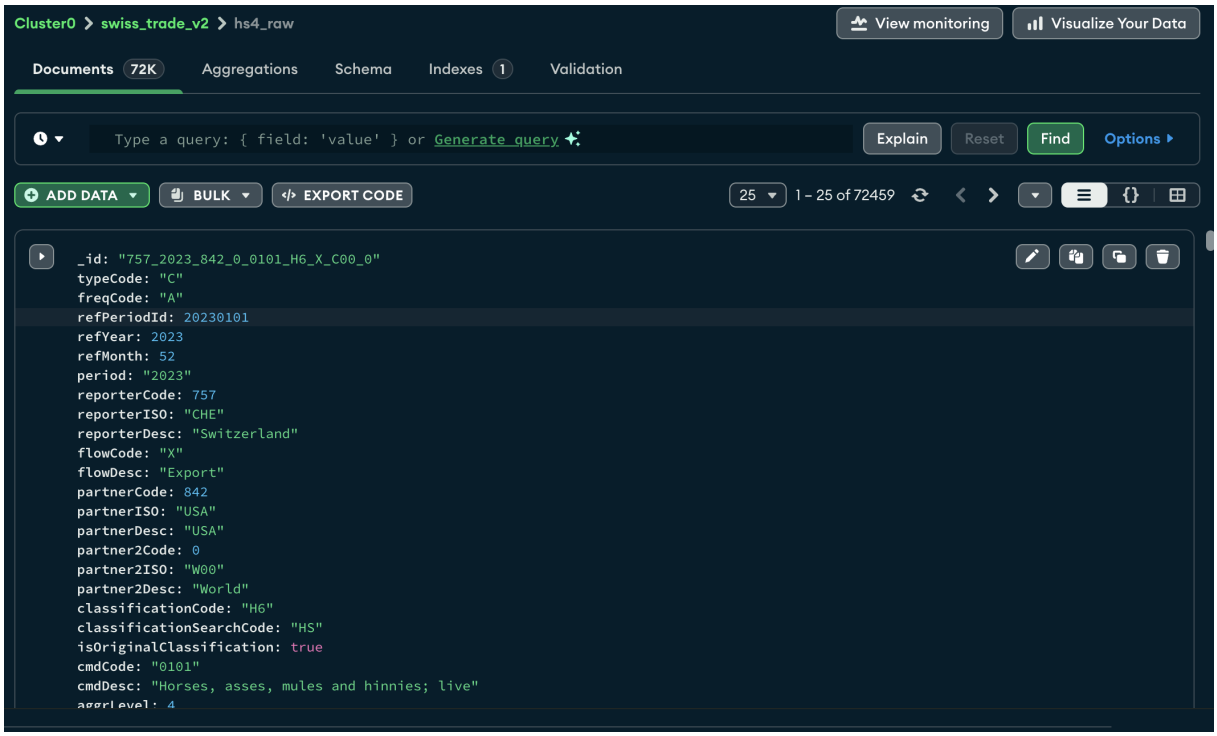


Figure 1: Atlas shows the HS4 raw collection and a 2023 Swiss export record for the USA.

Atlas shows 72,459 documents in `swiss_trade_v2.hs4_raw`, matching the verified raw load. Each document keeps 47 source fields plus our `_id`. These are raw observations, not the final

top-50 subset or the planned nested clean model. each document represents one specific trade observation.

3.11.2 Read the document identifier

The displayed `_id` is a string that our loading function creates:

```
757_2023_842_0_0101_H6_X_C00_0
```

Part	Source field	Meaning
757	reporterCode	Switzerland reports the trade
2023	period	Annual observation for 2023
842	partnerCode	USA is the primary partner
0	partner2Code	Aggregate second-partner dimension
0101	cmdCode	Live horses, asses, mules and hinnies
H6	classificationCode	HS 2022 edition
X	flowCode	Exports
C00	customsCode	Aggregate customs selection
0	motCode	Aggregate transport selection

MongoDB requires unique `_id` values within a collection. Our identifier differs from MongoDB's automatically generated `ObjectId`. Reruns use the same identifier to replace the same observation rather than create a duplicate.

Changing the year, partner, heading, or another identifier dimension produces a different `_id`. The HS code alone does not uniquely identify a trade observation.

3.11.3 Read the fields visible in the screenshot

The types below use Python terminology. JSON groups integers and floating-point values under its number type.

Field or related fields	Type	Displayed value	Meaning
_id	string	Identifier above	Unique observation key within this collection
typeCode	string	C	Goods rather than services
freqCode	string	A	Annual rather than monthly data
refPeriodId	integer	20230101	Source period identifier, not a shipment date
refYear	integer	2023	Numeric reference year
refMonth	integer	52	Source marker for annual data, not a calendar month

Field or related fields	Type	Displayed value	Meaning
`period`	string	`2023`	Year field used in our queries
`reporterCode`	integer	`757`	Comtrade code for the reporting country
`reporterISO`, `reporterDesc`	strings	`CHE`, `Switzerland`	Reporter ISO code and name
`flowCode`, `flowDesc`	strings	`X`, `Export`	Trade direction and description
`partnerCode`	integer	`842`	Comtrade code for the primary partner
`partnerISO`, `partnerDesc`	strings	`USA`, `USA`	Partner ISO code and source label
`partner2Code`	integer	`0`	Aggregate second-partner selection
`partner2ISO`, `partner2Desc`	strings	`W00`, `World`	Labels for that additional aggregate dimension
`classificationCode`	string	`H6`	HS 2022 edition, not a six-digit product level
`classificationSearchCode`	string	`HS`	Requested HS classification family
`isOriginalClassification`	boolean	`true`	Source uses its original classification rather than a converted edition
`cmdCode`	string	`0101`	Four-digit HS heading; preserve the leading zero
`cmdDesc`	string	`Horses, asses, mules and hinnies; live`	Description of that heading
`aggrLevel`	integer	`4`	Four-digit heading aggregation level

`partner2Desc = World` does not change the primary destination from USA to World. It describes the separate second-partner dimension, which we keep aggregated.

3.11.4 Measures and flags below the visible area

The saved HS4 snapshot contains these additional values for the same observation:

Field or related fields	Example value	Meaning
`primaryValue`, `fobvalue`	`6499937.824` each	Annual trade value in current US dollars; do not add these two fields together
`qty`, `qtyUnitAbbr`	`46.0`, `u`	Source quantity of 46 units, not 46 MongoDB documents

Field or related fields	Example value	Meaning
`netWgt`, `isNetWgtEstimated`	`23000.0`, `false`	Net weight in kilograms; the source does not mark it as estimated
`cifvalue`	`null`	No CIF value in this record; null does not mean zero
`isLeaf`, `isAggregate`, `isReported`	`false`, `true`, `false`	Source hierarchy and reporting flags for this heading-level total

`isReported=false` does not mean missing or failed data; Comtrade can construct heading totals from finer detail. This HS4 record contains a positive weight; earlier discovery totals had unusable placeholder weights. Check units and estimation flags per record before analyzing physical quantities.

```
top50_hs4 = []
top50_hs4_codes = []
if RANK_HS4_IN_MONGODB:
    if not CONNECT_TO_MONGODB:
        raise ValueError("Enable and verify the Atlas connection before ranking HS4 records.
    if HS4_RAW_COLLECTION == RAW_COLLECTION:
        raise ValueError("Rank HS4 detail in its separate raw collection.")
    validate_hs4_extract(hs4_records, hs4_extraction_log, hs4_query_scope)
    collection_hs4_raw = db[HS4_RAW_COLLECTION]
    verify_hs4_collection(collection_hs4_raw, hs4_records, hs4_query_scope)
    hs4_heading_codes = get_hs4_heading_codes(hs4_reference)
    validate_hs4_control_totals(hs4_control_totals, hs4_query_scope)
    ranking_filter = hs4_mongodb_filter(hs4_query_scope)
    ranking_filter["cmdCode"] = {"$in": sorted(hs4_heading_codes)}
    hs4_pipeline = [
        {"$match": ranking_filter},
        {"$group": {
            "_id": "$cmdCode",
            "commodityName": {"$first": "$cmdDesc"},
            "totalExportValue": {"$sum": "$primaryValue"},
            "observations": {"$sum": 1}
        }},
        {"$sort": {"totalExportValue": -1, "_id": 1}},
        {"$limit": 50}
    ]
    top50_hs4 = list(collection_hs4_raw.aggregate(hs4_pipeline))
    if len(top50_hs4) != 50:
        raise ValueError("Fewer than 50 HS4 headings are available; review the extraction sc
    top50_hs4_codes = [row["_id"] for row in top50_hs4]
    print(pd.DataFrame(top50_hs4).rename(columns={"_id": "hs4Code"}).to_string(index=False))
else:
    print("HS4 ranking skipped. Enable RANK_HS4_IN_MONGODB after verifying the raw load.")
```

`hs4Code`

7108
manufactured forms, or in powder form 4.082109e+11 84
3002 Human blood; animal blood for therapeutic, prophylactic or diagnostic uses; antiser
organisms (excluding yeasts) etc 1.633355e+11 90
3004 Medicaments; (not goods of heading no. 3002, 3005 or 3006) consisting of
2933
atom(s) only 9.253107e+10 90
9102
watches, pocket-watches, stop-watches and other watches, other than those of heading no. 910
7113
9101
watches, pocket-watches, stop-watches and other watches; with case of precious metal or of m
9021 Orthopaedic appliances; including crutches, surgical belts and trusses; splints and
2716
9018 Instruments
medical apparatus and sight testing instruments 1.189664e+10 90
0901
2935
2934
7106
manufactured forms, or in powder form 6.779763e+09 82
8479
8481
reducing valves and thermostatically controlled valves 6.181646e+09 90
2937 Hormones, prostaglandins, thromboxa
9027 Instruments and apparatus;
3302 Odoriferous substances and mixtures (including alcoholic solutions) with a basis of
8501
9701 Paintings, drawings and pastels
painted, hand-decorated manufactured articles; collages, mosaics and similar decorative plac
8419 Machinery, plant (not domestic), or laboratory equipment; electrically heated or
7110
manufactured forms, or in powder form 4.781258e+09 81
8802 Aircraft n.e.c. in h
9026 Instruments, apparatus for me
8414 Air or vacuum
tight biological safety cabinets whether or not fitted with filters 3.875113e+09
9031
2202 Waters, including mineral a
alcoholic beverages, not including fruit, nut or vegetable juices of heading no. 2009 3
7102
8504
3304 Cosmetic a
up and skin care preparations (excluding medicaments, including sunscreen or sun tan prepara
8536 Electrical apparatus for switching, protecting electr
2924
function compounds; amide-function compounds of carbonic acid 3.446360e+09 8
1806
8708
8535
8411

```
jets, turbo-propellers and other gas turbines      3.033887e+09            89
   8422
   7606
   3926
   8466 Parts & accessories suited for use only/mainly with machines of headings 8456-
8465, including work/tool holders, self-opening dieheads, dividing heads & other special att
   7112                              Waste and scrap of precious metal or of metal clad with
   8544                                                 Insulated wire, cable and other elect
   7318
pins, washers (including spring washers) and similar articles, of iron or steel      2.54340
   7103                                                  Precious (excluding diamond) and sem
precious stone; worked, graded, not strung, mounted, set; ungraded precious (excluding diamo
precious stone, temporarily strung for convenience of transport      2.508422e+09
   8483                          Transmission shafts (including cam and crank) and cranks; bea
   3923
   2941
   0406
   8413
```

```
[{'documents': 4235}]
```

3.11.5 Next stage: build the nested clean collection

The top 50 headings cover 85.72% of standard-heading export value within the selected 30 partner areas across 2023-2025. This percentage does not describe all Swiss exports. 3 years × 30 partners = 90 possible observations per HS4 code 90 × 50 headings = 4,500 possible observations

`observations` counts year-partner records, not shipments. Each heading can have at most 90 observations: 30 partners across three years. Electricity has 12 observations covering four partners across those years.

The selected subset contains **4,235 observations** out of 4,500 possible combinations. 4,500 represents all possible combinations—not combinations that must exist. We selected the top 50 headings overall, not headings that every partner imports from Switzerland.

```
pie showData
    title Selected year-partner-heading combinations
    "Existing observations" : 4235
    "Absent combinations" : 265
```

The next transformation will reshape these observations, not combine them into fewer documents:

```
flowchart TD
    RAW["hs4_raw: 72,459 flat documents"]
    RAW --> MATCH["Keep selected 50 headings: 4,235 observations"]
    MATCH --> SHAPE["MongoDB transformation: nest related fields"]
    SHAPE --> CLEAN["Planned trade_clean: 4,235 nested documents"]
```

- Keep one document per year, partner, and HS4 heading, with the other reporting dimensions fixed.
- Group fields into `reporter`, `partner`, `flow`, `commodity`, `trade`, and `quality` objects.
- Preserve both raw collections and do not add artificial zero-valued observations.
- Check the destination before writing; `$out` can replace an existing collection.

Expected clean counts are 1,416 for 2023, 1,412 for 2024, and 1,407 for 2025. After verifying the clean collection, continue with research-question pipelines and charts.

4 Sources and method references

- UN Comtrade API documentation: endpoints, query parameters, and authentication.
- UN Comtrade API guide: public preview versus authenticated access.
- Partner selection and World behavior: the `ALL` selection and the World aggregate.
- Subscription plans: current free-tier record, call, and rate limits.
- Partner-area reference: partner codes and descriptions.
- H6 classification reference: source codes for HS 2022 heading membership.
- WCO HS 2022 nomenclature: HS hierarchy and code counts.
- Local course notes: data modeling, PyMongo, and aggregation concepts.

Trade values are current US dollars and are not inflation-adjusted. Partner areas may include territories or aggregates. Source data may be revised.

5 PDF export and final submission checklist

The first Markdown cell contains Quarto YAML settings. Quarto supplies the title, table of contents, and section numbering. Rendering is configured not to execute Python cells; the PDF uses saved outputs.

Render V2 from this notebook's folder with:

```
quarto render "UN_Comtrade_V2.ipynb" --to pdf --no-execute
```

Before final submission:

- Complete the MongoDB connection, raw/clean transformations, analysis, conclusions, and learnings.
- Include at least two images, including a class diagram that matches the implemented clean model.
- Restart the kernel, run all intended cells successfully, and save outputs. Inspect API rerun and database write behavior first.
- Check for stale outputs, credentials, errors, clipped tables/code, and missing images.
- Render and inspect every PDF page, then package the PDF, executed V2 notebook, and required images in the ILIAS ZIP.

Do not treat this discovery-stage notebook or its partner-summary CSV as the completed analysis.